NATURAL LANGUAGE PROCESSING (NLP)

UNIT 5

LANGUAGE MODELING

- ✓ **Language Modelling**: Introduction, N-gram Models, Language Model Evaluation, Bayesian Parameter estimation, Language Model Adoption.
- ✓ **Language Models:** Class Based, Variable Length, Bayesian Topic-Based Language Model.
- ✓ Multilingual & Cross-Lingual Language Modeling.

Introduction

Many applications in human language technology involve the use of a statistical language model—a model that specifies the a priori probability of a particular word sequence in the language of interest. Given an alphabet or inventory of units and a sequence $W = w1w2....wt \in \Sigma^*$, a language model is used to compute the probability of W based on parameters previously estimated from a training set. Most commonly, the inventory (also called **vocabulary**) is the list of unique words encountered in the training data.

What is a Language Model?

Language Model (LM) is a probability distribution over sequences of words. In simpler terms, it's a statistical model that predicts the probability of a given sequence of words occurring in a language, or, more commonly, predicts the next word in a sequence based on the preceding words.

A language model calculates the probability of a given sequence of words (or other units) occurring in a specific language. In simple terms, it determines how "likely" or "natural" a sentence or phrase sounds.

OR

A Language Model is a probability distribution over sequence of words in a sentence.

In Simple terms a **Language Model is a statistical model**, that predicts the probability of a given sequence of words occurring in a language or it predicts the next word in a sequence based on the preceding word.

How it Works: It learns these probabilities from a large amount of text data, known as a training set.

Vocabulary (Σ): The model operates on a defined set of units, most commonly the unique words found in the training data, which is called the vocabulary.

.....

How are Language Models Used? (Applications)

OR Explain various applications of Language Model.

Language models are rarely used in isolation. They are typically combined with other models to improve the performance of larger systems. The following are the **Applications** of Language Model:

- ✓ **Speech Recognition**: An acoustic model might hear sounds that could be interpreted as "recognize speech" or "wreck a nice beach." A language model helps the system choose "recognize speech" because it is a much more probable sequence of words in English.
- ✓ **Machine Translation**: When translating a sentence, there can be many possible wordings. A language model helps select the most fluent and grammatically correct translation.
- ✓ **Information Retrieval** (Search Engines): Helps understand query intent and rank results. Eg: Google , Language Model Understands the query and retrieve relevant information by comparing the queries probability distribution.
- ✓ **Authorship Identification**: Analyzes text to determine who likely wrote it.

Optical Character Recognition (OCR): Helps decide the most likely sequence of characters when reading text from an image, correcting potential errors.

Spelling Correction: Language Model suggests likely corrections by comparing n-gram patterns of mistyped words.

Example: Correcting "receive" to "receive" using frequency-based bigrams.

Topic Modeling: N-grams help extract key phrases (e.g., "climate change", "data science") to identify topics in documents.

Language Identification: Distinguishes between languages (e.g., English vs. Spanish) based on frequent character or word n-grams.

---

n-gram Models

What is the need of n-Gram Model?

OR

Explain n-Gram Models.

The probability of a word sequence W of nontrivial length cannot be computed directly because unrestricted natural language permits an infinite number of word sequences of variable lengths. The probability P(W) can be decomposed into a product of component probabilities according to the chain rule probability:

$$P(W) = P(w_1 \ldots w_t) = P(w_1) \prod_{i=1}^{t} P(w_i | w_{i-1} w_{i-2} \ldots w_2 w_1)$$

Because the individual terms in this product are **still too difficult** to be computed directly, **statistical language models** make use of the **n-gram approximation**, also called as **n-gram models**. The assumption is that all previous words except for the n-1 words directly preceding the current word are irrelevant for predicting the current word or alternatively, that they are equivalent. Given the assumption of "*history equivalent classes*", the **n-gram model** is defined as:

$$P(W) \approx \prod_{i=1}^{t} P(w_i | w_{i-1}, \ldots w_{i-n+1})$$

Depending on the length of n, n-Gram model is implemented by using various models:

- ✓ **Uni Grams (n=1)**: The Probability of a word "w_i" is estimated based on the preceding word (w_{i-1}).
- ✓ **BiGram (n=2)**: The Probability of a word "w_i" is estimated based on the preceding word (w_{i-2}).
- ✓ **Tri Gram (n=3)**: The Probability of a word "w_i" is estimated only on the preceding words (w_{i-1})& (w_{i-2}).
- ✓ **4-grams and 5-grams** and so on.
- ✓ An **n-gram model is also called an (n-1)th order Markov Model**, because the approximation in Equation 2 represents the **Markov assumption** of the independent of the current word given all other words except the n-1 preceding words.

Advantages of n-Gram models:

The following are the various Advantages of **n-Gram models**:

✓ **To Predict the Next Word in the** sentence:

- **For Ex:** "I am going to the _____" → Based on past sequences, it might predict "store", "park", etc.

✓ **n-gram models assign Probabilities to Sentences.** It helps decide which sentence is more likely or correct, which is shown in the below example

- "He is reading a book." vs "He is read a book."
- The first one has higher probability under an n-gram model.

✓ **n-gram models simplifies Language Modeling :**

- Instead of considering the whole sentence, we use only n **previous words**. This reduces complexity.

✓ **n-gram models enable Autocomplete, Speech Recognition, Machine Translation:** Systems like Google Keyboard, Siri, or Google Translate use n-grams or advanced versions of it.

- ✓ n-Gram model is used in NLP to address the practical limitations of **chain rule of probability** when modeling language.
- ✓ An n-Gram model simplifies this by making the probability of a word depends only on the previous n-1 words, not the entire history.
- ✓ **Simple to Implement:** N-gram models are based on counting word sequences. No complex training is required.
- ✓ **Fast Computation:** They can be quickly trained and used, especially with small N values like unigram or bigram.
- ✓ **Effective for Short Contexts:** Useful when local word patterns (e.g., 2–3 words) are enough, such as in autocomplete or keyword detection.
- ✓ **Language Agnostic:** Works with any language without requiring deep grammar rules—just needs text data.
- ✓ **Good Baseline Model:** Often used as a baseline in NLP tasks to compare against more advanced models (like neural networks).
- ✓ **Probabilistic Output:** It provides **probability estimates** for next-word prediction or full sentence likelihood.

Example: For the sentence:

"I am going to the_____"

A **trigram model(n=3)** might compute the new word as shown below:

P("store"|"going to the")>P("banana"|"going to the")P(\text{"store"} \mid \text{"going to the"}) > P(\text{"banana"} \mid \text{"going to the"})P("store"|"going to the")>P("banana"|"going to the"), which helps predict **"store"** as a more likely next word.

.....

What are the limitations of the n-Gram models?

(or)

What are dis-advantages of the n-Gram models?

The following are the various Limitations of the N-gram model:

1. **Data Sparsity:**

   N-Gram model suffer when they are predicting a new word in the sequence of words, which have not seen in the Training data.

   Eg:"The quick brown fox" Appeared many types but 'The quick brown cat' never appear. The n-gram model assigns a zero probability to the word Cat, appearing after the quick brown

2. **Limited Context**

   N-gram models only consider the last N−1 words, missing long-term dependencies.

   Eg:"The book that I borrowed from the library was interesting."

   A **bigram model** would only look at pairs like:

   For EX: "borrowed from", "from the", "the library" ,etc.

It can't understand that "book" is related to "was interesting" because they are too far apart.

3. **Combinatorial Explosion:**

   As N increases, the number of possible N-grams grows rapidly, requiring more **memory and computation**.

4. **Fixed Window Size:**

   It can't dynamically adapt to context length like modern deep learning models (e.g., Transformers).

5. **Lack of Semantic Understanding:**

   N-gram models rely only on frequency and position, so they **don't understand meaning** or grammar.

6. **Poor Generalization:**

   If a sequence hasn't appeared in the training data, the model doesn't know how to handle it without smoothing techniques.

Language Model Evaluation

Explain the Language Model Evaluation Metrics.

- A language model computes the probability of a word sequence W.
- The below two criteria are used to evaluate a Language model:

1.Coverage Rate

2.Perplexity

1.Coverage Rate

- The coverage rate measures the percentage of n-grams in the test set that are represented in the language model.
  - A special case of this is the **out of vocabulary** rate (Or OOV rate), which is 100 minus the unigram coverage rate, or, in other words, the percentage of unique word toes not covered by the language model.
  - OOV Rate counts how many unique new words in the text data that were not in the original dictionary that is its vocabulary from training.
  - If the model encounters many words it has never seen before is considered as high OOV Rate, it will struggle to understand or predict sentences.

2. Perplexity:

- Perplexity is an information theoretic measure.
- Given a model of a discrete probability distribution, perplexity is defined as 2 raised to the entropy of p:

$$PPL(p) = 2^{H(p)} = 2^{-\sum_x p(x)\log_2 p(x)}$$

- In language modelling, the performance of a language model q on test set of a fixed size, say t words(w1,w2,....wt). then, language model perplexity is computed as:

$$PPL(p,q) = 2^{H(p,q)} = 2^{-\sum_{i=1}^{t} p(w_i)\log_2 q(w_i)}$$

Where q(wi) computes the probability of the ith word. If q(wi) is an n-gram probability, the equation becomes

$$2^{-\frac{1}{t}\sum_{i=1}^{t}\log_2 p(w_i|w_{i-1},\ldots,w_{i-n+1})}$$

When **comparing** different language models, especially models based on different ways of decomposing a text into language modeling units (e.g., words vs. morphemes), care must be taken to **normalize their perplexities** with respect to the same number of units in order to obtain a meaningful comparison.

Perplexity is defined as the average number of equally likely successor words when transitioning from one position in the word string to the next. If the model has no predictive power at all, perplexity is equal to the vocabulary size. A **model achieving perfect prediction**, by contrast, has a perplexity of one. The goal in language model development is most often to **minimize the perplexity** on a held-out data set representative of the domain of interest.

The goal of language modeling is not to predict word sequence probabilities but to **distinguish between** "good" and "bad" word sequence hypotheses generated by some frontend such as a machine translation system or a speech recognizer. In this case, the **language model** should assign **maximally distinct scores** to word sequences that are errorful, ungrammatical, or otherwise unacceptable, as opposed to those that are correct. Optimizing for minimum perplexity does not necessarily achieve this goal;

Conclusion:

- If a Language model is confused or surprised by the sentence, we can say that is having high perplexity and the language named is poor and the test sentence or surprising to it.
- If the Language model is really good it will find the sentence very predictable and will not be surprised much, which gives low perplexity.

Thus, Language Model which shows **low coverage** rate and **low perplexity** will be considered as good Language Model or else it will be considered as bad Language Model.

Parameter Estimation

Explain Bayesian Parameter Estimation.

Answer:

1. Introduction: The Core Concept

Bayesian Parameter Estimation is an alternative to standard estimation methods like Maximum Likelihood Estimation (MLE). Its fundamental difference is that it treats the set of model parameters (θ) itself as a **random variable**. This randomness is governed by a **prior probability distribution**, $P(\theta)$, which expresses our beliefs about the parameters *before* observing any data.

Given a training sample S, Bayesian estimation uses Bayes' rule to calculate the **posterior distribution** $P(\theta|S)$, which represents our updated beliefs about the parameters after seeing the data.

Equation: $P(\theta|S) = (P(S|\theta)P(\theta)) / P(S)$

In language modeling, θ represents the vector of word probabilities (e.g., $\theta = \{P(w_1), \ldots, P(w_K)\}$ for a unigram model), and S is the training sequence of words, $w_1, w_2, \ldots w_t$.

2. Point Estimation from the Posterior Distribution

Since the posterior $P(\theta|S)$ is a full distribution, we typically require a single "point estimate" for practical use. The text describes two main methods to obtain this:

A. Maximum a Posteriori (MAP) Estimation

The MAP estimate is the value of θ that maximizes the posterior probability. It finds the *mode* of the posterior distribution.

Equation: $\theta_{MAP} = \underset{\theta \in \Theta}{\operatorname{argmax}}\ P(\theta|S) = \underset{\theta \in \Theta}{\operatorname{argmax}} P(S|\theta)P(\theta)$

B. The Bayesian Criterion

This criterion finds the **expected value** (or mean) of the parameter θ given the sample S.

Equation: $\hat{\theta}_B = E[\theta|S] = \int_\theta \theta P(\theta|S) d\theta = \int_\theta \frac{P(S|\theta)P(\theta)}{P(S|\theta)P(\theta)} d\theta$

This can be expanded using the definition of the posterior distribution:

Equation; where θ is the space of all possible assignments for θ.

3. The Role of the Prior Distribution

The choice of the prior $P(\theta)$ is crucial and leads to different estimation functions.

A. The Uniform Prior

If we assume the prior distribution is uniform (i.e., all parameter values are equally likely), then:

- The **MAP estimate** becomes equivalent to the **Maximum-Likelihood Estimate (MLE)**.
- The **Bayesian estimate** ($\hat{\theta}_B$) becomes equivalent to the MLE with **Laplace smoothing**.

Equation : $\hat{\theta}_w^B = (c(w) + 1) / (\sum_{w'} c(w') + K)$

B. The Dirichlet Prior (The Most Common Choice)

The **Dirichlet distribution** is the most commonly used prior in language modeling.

- **Reason:** It is the **conjugate prior** for the multinomial distribution. This means that if the prior is a Dirichlet distribution, the posterior will also be a Dirichlet distribution, which simplifies calculations.
- **Formula:** The Dirichlet distribution is defined as:

  Equation (5.21):

  $$P(\theta) = D(\alpha_1, \ldots, \alpha) = \frac{\Gamma(\sum_{k=1}^{K} \alpha_k)}{\prod_{k=1}^{K} \Gamma(\alpha_k)} \prod_{k=1}^{K} \theta_k^{\alpha_k - 1}$$

- Here, $\alpha_1, \ldots, \alpha_K$ are **hyperparameters** that can be thought of as "counts" from a prior training sample.

- **MAP Estimate with Dirichlet Prior:** The MAP estimate under a Dirichlet prior results in an another Dirichlet distribution and is equivalent to MLE with **add-m smoothing**:

Equation (5.22):

$$\hat{\theta}^{MAP} = \underset{\theta \in \Theta}{\operatorname{argmax}} \frac{\Gamma\left(\sum_{k=1}^{K} \alpha_k\right)}{\prod_{k=1}^{K} \Gamma(\alpha_k)} \prod_{k=1}^{K} \theta_k^{n_k + \alpha_k - 1}$$

- This means we add pseudocounts of $m_k = \alpha_k - 1$ to each observed word count n_k.

4. Applications and Context

- **Language Model Adaptation:** The Bayesian approach using hyperparameters has been very successful for language model adaptation. The prior distribution is derived from a large out-of-domain dataset, and the posterior is calculated after observing a small in-domain dataset.
- **Evolution of Bayesian LMs:** Early Bayesian n-gram models did not perform as well as standard smoothed models like Kneser-Ney. However, recent progress, particularly in models that assume a latent topic structure (see Sec 5.6.8), has led to Bayesian techniques that yield results comparable to or better than traditional methods.

****

Language Model Adoption

Explain Language Model Adaption.

OR

Why Are Language Models Being Adopted?

OR

What is Language Model Adaptation and Why is it Needed?

1. Introduction: The Need for Language Model Adaptation

Language Model (LM) Adaptation is the process of designing and tuning a language model to perform well on a new, specific task or dataset for which there is insufficient training data.

This becomes necessary when a general-purpose language model, trained on a large but generic corpus, is applied to a new **domain** (e.g., legal documents), **topic** (e.g., financial

news), or **language**. In such cases, the vocabulary and statistical patterns of the new data differ significantly from the original training data, leading to poor performance. Adaptation aims to bridge this gap using a small amount of available specialized data.

2. Key Methods of Language Model Adaptation

Several methods have been developed to perform LM adaptation. The most prominent ones are:

A. Mixture Models (Model Interpolation)

This is the **most common** adaptation method. It works by combining a general model with a specific one.

- **Process:**
  1. An **in-domain model** is trained on a small amount of data specific to the target task (e.g., a few hundred medical articles).
  2. A **generic background model** is trained on a very large amount of out-of-domain data (e.g., a massive news corpus).
  3. These two models are **interpolated** by assigning them weights. The final probability of a word is a weighted average of the probabilities given by each model.
  4. The interpolation weights are optimized on a small development set to find the best balance between the general and specific models.

B. Topic-Dependent Adaptation

This approach adapts the model based on the underlying topic of the text.

- **Clustering-Based Models:** Documents are first clustered into different topics. Individual LMs are then built for each topic cluster. The final adapted model is created by choosing and interpolating a small number of these topic-specific LMs relevant to the current text.
- **Trigger Models:** This is a form of dynamic self-adaptation. The idea is that certain words can "trigger" other related words, making them more likely to appear. For

example, in financial news, the words stock and market trigger a higher probability for other finance-related terms. Underlying techniques used to identify these trigger relationships include:

- **Latent Semantic Analysis (LSA):** Uses Singular-Value Decomposition (SVD) on a document-word matrix to find semantic similarities between words.
- **Probabilistic Latent Semantic Analysis (PLSA):** A more advanced probabilistic approach that uses a latent class model. A known issue with PLSA is its tendency to overfit the training data.
- **Latent Dirichlet Allocation (LDA):** A more recent method that is considered a regularized version of PLSA, which helps prevent the overfitting problem.

z. Maximum a Posteriori (MAP) Adaptation

MAP adaptation is an alternative to mixture models and is considered **more robust** to variations in the adaptation data. Instead of interpolating model probabilities, it combines the raw frequency *counts* from the in-domain and out-of-domain datasets.

- **Formula:** The probability of a word w given a history h is calculated as:

$$P(w|h) = \frac{C_{OD}(w,h) \cdot \epsilon C_{ID}(w,h)}{C_{OD}(h) \cdot \epsilon C_{ID}(h)}$$

- **Where:**
  - C_{OD} is the count from the **Out-of-Domain** (generic) data.
  - C_{ID} is the count from the **In-Domain** (specific) data.
  - ϵ is a weighting parameter set empirically to balance the influence of the two datasets, as the out-of-domain data is usually much larger.

D. Unsupervised Adaptation

This technique is primarily used in **speech recognition**. Instead of using perfectly transcribed text for adaptation, it uses the direct output of a speech recognizer. Although this data is noisy and imperfect, studies show it can achieve roughly half the improvement gained from using perfect transcriptions.

E. Using the Internet for Data

When insufficient text data is available for a specific domain, the Internet can be used as a resource. The process involves:

1. Querying the web for relevant documents.
2. Retrieving, post-processing, and filtering the data.
3. This new data is then either added to the existing training data or used to build a separate model that is interpolated with the baseline LM.

3. Applications of Language Model Adaptation

The text highlights two main application areas:

1. **Speech Recognition:** This is a major field for LM adaptation, using techniques like unsupervised adaptation to tune models to specific speakers or acoustic environments.
2. **Machine Translation:** LMs can be adapted to the target language. For example, queries can be constructed from initial, rough translations to select relevant sentences from a large target-language corpus. This new data is then used to retrain or interpolate the baseline LM, improving translation fluency.

Conclusion:

Language model adaptation is a crucial set of techniques in NLP that allows general models to be specialized for new tasks with limited data. The primary methods involve combining general and specific models (Interpolation), adapting to the topic (Trigger Models, LDA), directly adjusting data counts (MAP), or leveraging new data sources (Unsupervised Adaptation, Internet). Of these, MAP adaptation is noted for being particularly robust.

Language Models

Explain the types Language Models.

Language Model: A Language Model is a probability distribution over sequence of words in a sentence.

In Simple terms a Language Model is a statistical model, that predicts the probability of a given sequence of words occurring in a language or it predicts the next word in a sequence based on the preceding words

The following are the types of **Language Models:**

1. Class Based Language Models
2. Variable length Language Models
3. Bayesian Topic Based Language Models

Class Based Language Models

1. Introduction: Purpose and Core Idea

Class-based language models are a simple and t effective method for addressing the problem of **data sparsity** in language modeling. The core idea is to group words with similar grammatical or semantic properties into "classes" and then build a statistical model based on these classes rather than individual words.

By modeling the probability of class sequences, the model can make reasonable predictions even for word sequences that were never seen in the training data, as long as the corresponding class sequence is plausible.

2. Creating Word Classes

Words can be clustered into classes in two primary ways:

- **Automatic Means:** Using statistical clustering algorithms to group words that appear in similar contexts.
- **Linguistic Criteria:** Using pre-defined linguistic categories, such as **Part-of-Speech (POS)** classes (e.g., all nouns in one class, all verbs in another).

3. The Statistical Model and Key Assumptions

The fundamental assumption of a class-based model is that **words are conditionally independent of other words, given their class**. This simplifies the modeling process significantly.

For a **class-based bigram model**, the probability of a word w_i given the preceding word w_{i-1} can be defined by summing over all possible class pairs (c_i, c_{i-1}), where c_i is the class of word w_i and it is shown as below:

$$P(w_i|w_{i-1}) = \sum_{c_i, c_{i-1}} P(w_i|c_i)\, P(c_i|c_{i-1}, w_{i-1})\, P(c_{i-1}|w_{i-1})$$

$$= \sum_{c_i, c_{i-1}} P(w_i|c_i)\, P(c_i|c_{i-1})\, P(c_{i-1}|w_{i-1})$$

This model is then simplified under the assumption that the current class c_i is independent of the previous word w_{i-1} once the previous class c_{i-1} is known This changes the middle term, such that the model simplifies to:

$$P(w_i|w_{i-1}) = P(w_i|c_i)\, P(c_i|c_{i-1})$$

1. P(w_i|c_i): The probability of a specific word w_i occurring, given that we know it belongs to class c_i. (Emission Probability)
2. P(c_i|c_{i-1}): The probability of transitioning from the previous class c_{i-1} to the current class c_i. (Class Transition Probability)

4. An Alternative Model and Performance

A more complex model proposed by **Goodman** conditions the current word not only on its own class but also on the preceding class.

$$P(w_i|w_{i-1}) \approx P(w_i|c_i, c_{i-1})\, P(c_i|c_{i-1})$$

- **Key Difference:** The term P(w_i|c_i, c_{i-1}) makes the word's probability dependent on the class bigram (c_i, c_{i-1}), not just its own class c_i.
- **Experimental Results:** Experiments on a large news corpus showed that this more complex model (Eq. 5.32) performed better than the simpler models, **unless the training data size was at the lower end**. This highlights a classic trade-off: more complex models can capture more detail but require more data to be trained effectively.

5. Advantages and Limitations

- **Advantage:** Class-based models are successful in **reducing perplexity** and improving the practical performance of language processing systems, especially by mitigating the data sparsity issue.
  - **Limitation:** For optimal performance, class-based models typically need to be **interpolated with a standard word-based language model**. They are used in combination with other models rather than as a complete replacement. The effectiveness of this model is depend on the quality of word clustering.

Variable Length Language Models

1. Introduction: Overcoming the Limitations of Standard LMs

Standard language models operate on two simple principles:

1. **Vocabulary units** are defined by whitespace (i.e., they are single words).
2. The probability of the next word is predicted based on an invariable, fixed-length history (e.g., the previous two words in a trigram model).

Variable-Length Language Models (VLLMs) are a modification of this standard approach. Their goal is to redefine the vocabulary in a **data-driven way** by creating **merged units** (i.e., phrases) composed of a variable number of basic words. These models are also referred to as **variable-length n-gram models**.

2. The Core Challenge

The main challenge in building a VLLM is two-fold:

1. **Segmentation:** Finding the best way to segment a sequence of words ($w_1 w_2 \ldots w_t$) into a new sequence of language modeling units (which can be single words or multi-word phrases).
2. **Probability Estimation:** Estimating the probabilities for this new set of variable-length units.

3. Approaches to Creating Variable-Length Models

The text describes a few different approaches to tackle this challenge:

A. The Complex Approach: Segmentation as a Hidden Variable

- In this method, the segmentation of the text is treated as a hidden variable.
- A technique like the Maximum Entropy (ME) procedure is used to find the optimal segmentation.
- **Result:** A variable-length model of order 7 showed only **slight improvements in perplexity** over a standard word-based bigram model, and no practical application results were reported.

B. The Simpler Approach: Greedy Merging of Units

This is a more straightforward and common method that does not attempt to re-segment the entire vocabulary from scratch.

- **Process:**
  1. Start with the standard segmentation based on whitespace (i.e., individual words).
  2. Identify potential phrasal candidates to be merged. A common criterion for this is the **mutual information** between adjacent words. High mutual information suggests that two words frequently appear together and may form a meaningful unit (e.g., "New York").
  3. Use a **greedy iterative algorithm** to select the final phrasal units. In each step of the algorithm, the candidate phrase that reduces the perplexity of a development corpus the most is chosen and added to the vocabulary.
  4. The final language model vocabulary contains both the original words and a limited number of these newly formed short phrases.

C. An Advanced Variant: Using Word Class Information

This approach refines the greedy merging method by incorporating word class information.

- **Process:** Instead of identifying candidate phrases by calculating mutual information between pairs of words, it is calculated over **pairs of classes**. This allows the model to generalize across words that belong to the same category.
- **Results:** This method was shown to be highly effective:
  - It **reduced perplexity by roughly 10%** compared to selecting candidate pairs based on words alone.
  - It achieved an **18% relative reduction in word-error rate (WER)** on a medium-scale French Automatic Speech Recognition (ASR) task, demonstrating significant practical benefits.

Conclusion:

Variable-Length Language Models enhance standard LMs by moving from fixed-length histories of single words to variable-length units that include common phrases. While complex methods exist, a simpler and effective approach involves greedily merging adjacent words based on mutual information. Using word class information to guide this merging process has been shown to yield significant improvements in both perplexity and real-world application performance, such as in ASR.

Bayesian Topic-Based Language Models

1. Introduction

Bayesian Topic-Based Language Models represent a significant recent trend in language modeling that focuses on incorporating the latent (hidden) topic structure of documents. The core idea is to model a document as a mixture of several topics, where each topic is a probability distribution over words. One of the first and most prominent models of this type is **Latent Dirichlet Allocation (LDA)**.

2. Latent Dirichlet Allocation (LDA) Model

The LDA model, proposed by Blei, Ng, and Jordan, provides a generative model for documents. It makes the following key assumptions:

- A document is a collection of topics.
- Each topic is a distribution over words.
- A document is treated as a **"bag of words"**, meaning the order of words (n-grams) is not taken into account in the basic model.

The Generative Process:

The process by which LDA assumes a document is generated is as follows:

1. The topics themselves (θ_1, θ_2, ..., θ_K) are assumed to be drawn from a **Dirichlet distribution**, which is governed by hyperparameters α_1, ..., α_K.
2. Each topic k is represented by a probability distribution over words, denoted by φ_k.
3. To generate a word in a document:
   - First, a topic z_k is chosen based on the document's topic distribution θ_k.
   - Then, a word w is sampled from that chosen topic's word distribution $\varphi_{k(w)}$.

Key Formulas: The probability distribution for the set of K topics is given by the Dirichlet distribution:

$$P(\theta_1, \ldots, \theta_K) = \frac{\Gamma(\sum_k \alpha_k)}{\prod_k \Gamma(\alpha_k)} \prod_{k=1}^{K} \theta_k^{\alpha_k - 1}$$

- The probability of an entire document w (consisting of t words) is modeled by integrating over the latent topic variables:

$$p(w|\alpha, \varphi) = \int p(\theta|\alpha) \left(\prod_{i=1}^{t} \sum_{z_i} p(z_i|\theta) p(w_i|z_i, \phi) \right) d\theta$$

3. Challenges and Practical Implementation

- **Inference Challenge:** The main challenge of LDA is computing the posterior distribution of the latent variables (θ and z). This is not analytically tractable, so **approximation**

techniques are required, such as **Markov chain Monte Carlo (MCMC)** sampling or **variational inference**.

- **Limitation and Combination with n-grams:** Because the basic LDA model is a **unigram model**, for practical use in language modeling, it must be combined with a standard n-gram model.
  - **Example 1:** Wang et al. combined LDA with a trigram and a probabilistic context-free grammar. This yielded perplexity reductions of **9% to 23%** on the Wall Street Journal corpus.
  - **Example 2:** Hsu and Glass used LDA with a hidden Markov model for speech recognition. This achieved a **16.1% perplexity reduction** and a **2.4% word-error rate reduction** over an already adapted trigram model.

4. Advanced Extensions of Bayesian Topic Models

The LDA model has been extended in several powerful ways:

- **Non-parametric Models (Dirichlet Processes):** LDA can be generalized to use **Dirichlet processes**, which allows the model to handle an **infinite number of topics**. This removes the need to pre-specify the number of topics (K), allowing it to be adjusted based on the training data.
- **Hierarchical Dirichlet Process (HDP):** This extension allows topics to be structured hierarchically (e.g., a topic can have subtopics) and shared between different data clusters.
  - **Application:** Huang and Renals used HDPs for meeting-style conversational speech recognition. This yielded **slight word-error-rate reductions (0.3% absolute)**.
- **Pitman-Yor Process:** This is another type of Bayesian model that has shown significant promise. A language model based on a Pitman-Yor process can, **by itself**, achieve perplexities comparable to a high-performing **Kneser-Ney smoothed trigram model** without requiring interpolation with a baseline model. This is a major advantage as it simplifies the overall system.

Multilingual and Cross lingual Language Modeling

Q: Compare and contrast Multilingual and Cross-lingual Language Models.

1. Introduction

Multilingual and Cross-lingual Language Modeling are two advanced approaches that address challenges arising from the interaction between different languages. While both deal with more than one language, they solve fundamentally different problems.

- **Multilingual Language Modeling** aims to build systems that can process input containing multiple languages, either presented sequentially or mixed together within a single utterance (a phenomenon known as code-switching).
- **Cross-lingual Language Modeling** focuses on leveraging data from one language (a source language) to improve the performance of a language model for a different language (a target language), particularly when the target language has insufficient data.

2. Comparison and Contrast

Feature	Multilingual Language Modeling	Cross-lingual Language Modeling
Goal	To handle and process input that is inherently multilingual. The system must recognize and understand multiple languages simultaneously or sequentially.	To improve a language model for a data-scarce target language by transferring knowledge from a data-rich source language.
Problem Solved	Addresses real-world scenarios like **code-itching** (e.g.,"Spanglish") or applications used by speakers of different languages (e.g., a telephone-based dialog system).	Addresses the **data sparsity** problem for a specific language by "borrowing" statistical information from another, often related, language.

Key Techniques	1. **Dynamic Switching:** A first-step language identification system determines the input language, and the system switches to the appropriate monolingual model 2. **Model Combination:** Monolingual corpora are combined to train a single, larger multilingual model. This can be done by simply pooling the data or using more complex methods like context-free grammars to manage language states. 3 **Common Backoff:** For handling intrasentential code-switching, a common backoff node can be introduced in the model to allow for language switches after a pause.	1. **Automatic Translation:** The most straightforward method is to automatically translate text from the source language into the target language and use this (errorful) translated text as additional training data. 2. **Dictionary-Based Translation:** Instead of full machine translation, a high-quality bilingual dictionary (extracted from parallel corpora) is used to generate word-by-word translations, which are then used to build or enhance the LM. 3. **Bilingual Latent Semantic Analysis (bLSA):** An advanced adaptation method that projects words from both languages into a common semantic space, enforcing a one-to-one topic correspondence. This allows topic probabilities from the source language to directly inform the target language model.
Example Use Case	- A speech-based information kiosk that serves speakers of different languages. Modeling utterances that mix	- Improving a Mandarin news LM by incorporating automatically translated English news text. Developing a model for Icelandic

	Spanish and English within the same sentence.	weather reports using a small amount of in-domain data and a larger parallel English-Icelandic corpus.
Underlying Data	Requires corpora from all languages the final system needs to support. The focus is on the *breadth* of languages handled.	Requires data from the source language and, for better results, a parallel corpus or bilingual dictionary that links the source and target languages. The focus is on the *transfer* of information.

3. The bLSA Approach in Cross-lingual Modeling

A key recent development in cross-lingual modeling is the bLSA adaptation. It works by determining the topic-dependent word probabilities in the target language based on topic priors from the source language.

The probability of an English word w can be influenced by Chinese topic priors θ_{k}^{Ch} as follows:

Equation

$$P_{En}(w) = \Sigma_{k} \varphi_{k}^{En}(w) \theta_{k}^{Ch}$$

This adapted probability is then used to update the baseline target model:

Equation :

$$P_{target}(w|h) \propto (P_{bLSA}(w) / P_{base}(w))^{\beta} * P_{base}(w|h)$$

This method was shown to reduce perplexity by 9-13% and improve the BLEU score on a machine translation task.

4. Conclusion: Key Differences Summarized

- **Focus:** Multilingual modeling is about **input handling** (reacting to multiple languages), whereas cross-lingual modeling is about **model improvement** (using one language to build a better model for another).
- **Direction of Information:** In multilingual models, information about different languages co-exists. In cross-lingual models, information flows *from* a source language *to* a target language.
- **Primary Scenario:** Multilingual models are for applications that are designed to be used in multilingual environments. Cross-lingual models are a solution for building effective models for low-resource languages.

Q: Explain Multilingual and Cross-lingual Language Models.

Answer:

1. Introduction

Multilingual and Cross-lingual language modeling are advanced techniques that extend traditional language models to handle scenarios involving more than one language. While both deal with multiple languages, they address distinct problems:

- **Multilingual models** are designed to process input that contains multiple languages.
- **Cross-lingual models** are designed to leverage data from one language to build a better model for another language.

2. Multilingual Language Modeling

Purpose:

The primary goal of multilingual language modeling is to create systems that can function correctly when presented with input from multiple languages. This can happen in two main ways:

1. **Sequentially:** When a system serves different users speaking different languages (e.g., a telephone-based dialog system).

2. **Simultaneously:** In the case of **code-switching**, where speakers mix multiple languages or dialects within a single utterance. An example is "Spanglish," such as the sentence: *"I need to tell her que no voy a poder ir."*

Techniques:

Several methods are used to build multilingual models:

- **Dynamic Language Switching:** A first-step language identification system determines the language of the input. The system then dynamically switches to the appropriate monolingual model to process the utterance.
- **Model Combination:** Different monolingual corpora are combined into a single, larger multilingual model. This can be done by simply pooling the data or by using more structured approaches, such as a context-free grammar where nonterminal states can expand to monolingual n-grams.
- **Handling Intrasentential Switching:** To specifically model code-switching within a sentence, a common **backoff node** can be introduced, which allows for a language switch to occur after a pause has been detected.

Limitations:

- The text notes that building a model for intrasentential (within-sentence) code-switching is **more difficult** than handling sequential language changes.
- Simply pooling data to create a combined model can **degrade performance**, especially when the sizes of the individual language corpora are unbalanced.

3. Cross-lingual Language Modeling

Purpose:

The goal of cross-lingual language modeling is to address data sparsity by leveraging information from a data-rich language (source language) to improve a language model for a data-scarce language (target language).

Techniques: Several methods have been developed for this knowledge transfer:

- **Automatic Translation:** This is the most straightforward approach. Text from the source language is automatically translated into the target language. This translated

text, though often errorful, is used as additional training data. For example, a Mandarin news LM was improved by enriching its training data with automatically translated English news text.

- **Dictionary-Based Approach:** A high-quality bilingual dictionary, extracted from document-aligned parallel corpora, is used to create word-for-word translations. A unigram model built from these translations is then interpolated with the baseline target LM. This avoids the need for a full, complex machine translation system.
- **Crosslingual Semantic Analysis (cLSA):** This method projects words from both the source and target languages into a common semantic space, which allows for the construction of word translation probabilities.
- **Bilingual Latent Semantic Analysis (bLSA):** This is an advanced adaptation approach. It uses a separate LSA model for each language but enforces a one-to-one topic correspondence. Topic distributions from the source language are used to influence the topic-dependent word probabilities in the target language.

Formulas for bLSA:

The topic-dependent word probability `P_En(w)` in the target language (English) is determined using topic priors `θ_k^Ch` from the source language (Chinese):

$$P_En(w) = \sum_k \phi_k^{En}(w)\theta_k^{Ch}$$

This adapted probability `P_bLSA` is then incorporated into the target model:

$$P_target(w|h) \propto \left(\frac{P_{bLSA}(w)}{P_{base}(w)}\right)^{\beta} P_{base}(w/h)$$

Advantages and Results:

- Cross-lingual methods have shown significant improvements. The automatic translation approach improved perplexity by about **10%** and speech recognition word-error rate by **0.5% absolute**.
- The bLSA approach reduced perplexity by **9% to 13%** and improved the BLEU score by up to 0.3 points on a machine translation task.

Limitation: A major drawback of approaches that rely on translation is that the quality of the resulting model is **heavily dependent on the translation accuracy**.

4. Conclusion

In summary, Multilingual and Cross-lingual LMs serve different purposes. Multilingual models focus on **handling input that is already mixed-language**, enabling applications like code-switching detection or multilingual dialog systems. In contrast, Cross-lingual models focus on **improving a single-language model** by transferring statistical knowledge from another language, which is especially valuable for low-resource languages.

******

IMPORTANT QUESTIONS

1) What is Language Model? Explain various applications of Language Model.
2) Explain n-Gram Models. (Including Advantages & Disadvantages (Limitations))
3) Explain the Language Model Evaluation Metrics.
4) Explain the types of Language Models (All 3 Models).
5) Compare and contrast Multilingual and Cross-lingual Language Models.